HDC-RWKV (nb12c): Every Notation, Every Flow

A worked example for training and inference

Wozformer paper appendix

This document accompanies the architecture diagram (`architecture_hdc_rwkv.tex`). It defines every tensor and operator, then traces a complete training step and a complete inference step through the shipping model on a deliberately tiny instance (V=4, d=4, T=3, L=1) so every number is verifiable by hand. The shipping configuration is V=256, d=256, L=1, identical in every operation; only the dimensions change.

Contents

1 Conventions and symbols **1**

2 The four learnable tensors **2**

3 Auxiliary operators **3**
- 3.1 Element-wise sign with straight-through estimator . . . 3
- 3.2 Cyclic right-rotation ρ^t (HDC binding) . . . 4
- 3.3 Hyperbolic tangent . . . 4
- 3.4 Cross-entropy loss . . . 4

4 The forward pass — definitions **4**

5 Worked training step on a toy instance **5**
- 5.1 Step 1 — Token lookup and rotation . . . 6
- 5.2 Step 2 — Recurrence . . . 6
- 5.3 Step 3 — Logits and loss at the final position . . . 6
- 5.4 Step 4 — Backward pass through STE . . . 7
- 5.5 Recap of the full training step . . . 7

6 Worked inference (autoregressive sampling) **7**

7 What the 6502 actually computes **8**

8 Cross-reference to the source **9**

1 Conventions and symbols

Indices and shapes

Symbol	Meaning	Range / shape
B	batch size (number of independent sequences)	$\mathbb{N}_{>0}$
T	sequence length (block size)	$\mathbb{N}_{>0}$
V	vocabulary size (number of BPE tokens)	$\mathbb{N}_{>0}$
d	hypervector dimension	$\mathbb{N}_{>0}$
L	number of recurrent layers (L=1 in nb12c)	$\mathbb{N}_{>0}$
t	time step (token position) within a sequence	$0, 1, \ldots, T-1$
b	batch index	$0, 1, \ldots, B-1$
v	vocabulary index	$0, 1, \ldots, V-1$
i	hypervector channel index	$0, 1, \ldots, d-1$
$x_t^{(b)}$	input token id at position t in sequence b	$\{0, \ldots, V-1\}$
$y_t^{(b)}$	target token (the answer for position t)	$\{0, \ldots, V-1\}$

Sets and spaces

- $\{-1, +1\}^d$ — the bipolar hypercube. Vectors of length d whose entries are exactly -1 or $+1$. The natural state space of HDC.
- $(-1, +1)^d$ — the open hypercube, image of tanh. The *continuous* state space we use for the recurrent activation.
- $\mathbb{R}^{V \times d}$ — the continuous training-time storage for the bipolar tensors. At inference these tensors are reduced to a single bit per entry; at training they hold real-valued scratch values that get projected to $\{-1, +1\}$ through STE on every forward pass.

2 The four learnable tensors

The model has exactly four learnable objects. The first three are bipolar (at inference); the fourth is a single scalar.

Learnable parameters

Name (code)	Symbol	Shape (training)	Role
`vocab_hv_c`	$\mathbf{V}_{hv} \in \mathbb{R}^{V \times d}$	$V \times d$	Embedding table. Row v is the *token hypervector* of vocabulary item v. Used through STE so the embedding seen by the recurrence is $\operatorname{sign}(\mathbf{V}_{hv}) \in \{-1, +1\}^{V \times d}$.
`prototype_hv_c`	$\mathbf{P}_{hv} \in \mathbb{R}^{V \times d}$	$V \times d$	Output "cleanup" matrix. Row v is the *prototype hypervector* for token v; the logit for token v is the inner product of the state with $\operatorname{sign}(\mathbf{P}_{hv})_v$.
`decay_masks_c[0]`	$\mathbf{m} \in \mathbb{R}^{d}$	d	Per-channel recurrence gate. After STE it lives in $\{-1, +1\}^d$ and tells each channel whether to keep $(+1)$ or flip (-1) its previous state on each step.
`log_temp`	$\log \tau \in \mathbb{R}$	scalar	Output temperature, stored in log-space so $\tau > 0$ is automatic. Initialised to $\log \sqrt{d}$.

Deployment storage cost

After training, only the *sign* of each bipolar tensor matters. We pack 8 bits per byte:

$$\text{bytes}_{\text{deploy}} = \underbrace{\tfrac{V \cdot d}{8}}_{\mathbf{V}} + \underbrace{\tfrac{V \cdot d}{8}}_{\mathbf{P}} + \underbrace{\tfrac{L \cdot d}{8}}_{\text{decay}} + 2 \ (\tau \text{ in fp16}).$$

For nb12c (V=256, d=256, L=1): $8192 + 8192 + 32 + 2 \approx 16{,}418$ bytes; the on-disk file adds a small header, totalling 16,434 bytes.

3 Auxiliary operators

Four operators appear in the forward pass; this section defines each.

3.1 Element-wise sign with straight-through estimator

$\widetilde{\text{sign}}$ — the operator that makes the model trainable

The function $\widetilde{\text{sign}} : \mathbb{R}^n \to \{-1, +1\}^n$ has the *forward* value

$$\widetilde{\text{sign}}(x)_i = \begin{cases} +1, & x_i \geq 0 \\ -1, & x_i < 0 \end{cases}$$

and the *backward* (subgradient) value

$$\frac{\partial \, \widetilde{\text{sign}}(x)_i}{\partial x_i} = \begin{cases} 1, & |x_i| \leq 1 \\ 0, & |x_i| > 1 \end{cases} \qquad \text{("clipped STE").}$$

In code this is realised by the algebraic identity

$$\widetilde{\text{sign}}(x) \;=\; \underbrace{\text{sign}(x).\text{detach}()}_{\text{forward value}} \;+\; \underbrace{\text{clamp}_{[-1,1]}(x) - \text{clamp}_{[-1,1]}(x).\text{detach}()}_{\text{zero forward, identity gradient on } [-1,1]}.$$

The detached terms have value but no gradient; the non-detached $\text{clamp}_{[-1,1]}(x)$ supplies the gradient. See `wozformer/ste.py`.

3.2 Cyclic right-rotation ρ^t (HDC binding)

ρ^t — position binding via permutation

For a hypervector $\mathbf{h} \in \mathbb{R}^d$ and shift $t \in \mathbb{Z}$,

$$\rho^t(\mathbf{h})_i \;=\; \mathbf{h}_{(i-t) \bmod d}.$$

Equivalent to `torch.roll(h, shifts=t, dims=-1)`. Key properties:

1. *Preserves bipolarity*: $\rho^t(\{-1,+1\}^d) = \{-1,+1\}^d$.
2. *Preserves quasi-orthogonality*: $\langle \rho^a(\mathbf{h}), \rho^b(\mathbf{h}) \rangle = d$ iff $a \equiv b \pmod d$, and is small otherwise.
3. *Composes additively*: $\rho^a \circ \rho^b = \rho^{a+b}$.

Item (2) is why ρ^t replaces a positional embedding table: positions $t_1 \neq t_2$ produce vectors that look uncorrelated to the recurrence, but the operation is parameter-free.

3.3 Hyperbolic tangent

Bounded continuous activation $\tanh : \mathbb{R}^d \to (-1,+1)^d$, elementwise. Used to keep the recurrent state finite without binarising it (finding F10 in `findings.md` shows binarising the state destroys training).

3.4 Cross-entropy loss

For one position with logits $\mathbf{z} \in \mathbb{R}^V$ and integer target $y \in \{0,\dots,V-1\}$,

$$\text{CE}(\mathbf{z}, y) \;=\; -\log\left(\frac{e^{z_y}}{\sum_{v=0}^{V-1} e^{z_v}}\right) \;=\; \log\Big(\sum_v e^{z_v}\Big) - z_y.$$

Averaged over all $B \cdot T$ positions.

4 The forward pass — definitions

We give the forward pass first in symbolic form. §5 then executes every line on the toy instance.

Inputs. A batch of token-id sequences $\mathbf{x} \in \{0,\dots,V-1\}^{B\times T}$ and (for training) targets $\mathbf{y} \in \{0,\dots,V-1\}^{B\times T}$ where typically $y_t = x_{t+1}$.

Step 1 — Materialise bipolar tensors.

$$\widetilde{\mathbf{V}} \;=\; \widetilde{\text{sign}}(\mathbf{V}_{hv}) \;\in\; \{-1,+1\}^{V\times d}, \tag{1}$$

$$\widetilde{\mathbf{P}} \;=\; \widetilde{\text{sign}}(\mathbf{P}_{hv}) \;\in\; \{-1,+1\}^{V\times d}, \tag{2}$$

$$\widetilde{\mathbf{m}} \;=\; \widetilde{\text{sign}}(\mathbf{m}) \;\in\; \{-1,+1\}^{d}. \tag{3}$$

Step 2 — Token lookup and position binding. For each position $t \in \{0, \dots, T-1\}$ and each batch element b,

$$\mathbf{h}_t^{(b)} \;=\; \rho^t\big(\,\widetilde{\mathbf{V}}_{\,x_t^{(b)}}\,\big) \quad \in\; \{-1,+1\}^d.$$

This produces a (B, T, d) position-tagged bipolar tensor.

Step 3 — Linear recurrence with continuous state. For each batch element, initialise $\mathbf{s}_{-1}^{(b)} = \mathbf{0} \in \mathbb{R}^d$ and iterate

$$\boxed{\mathbf{s}_t^{(b)} \;=\; \tanh\Big(\,\widetilde{\mathbf{m}} \odot \mathbf{s}_{t-1}^{(b)} \;+\; \mathbf{h}_t^{(b)}\,\Big)} \qquad t = 0, 1, \dots, T-1,$$

where $\odot$ is element-wise multiplication. Note: $\widetilde{\mathbf{m}} \in \{-1,+1\}^d$, $\mathbf{s}_{t-1} \in (-1,+1)^d$, $\mathbf{h}_t \in \{-1,+1\}^d$, so each channel of the tanh-input lies in $[-2,+2]$ and the state stays in $(-1,+1)^d$ by construction.

Step 4 — Prototype-similarity logits. With $\tau = e^{\log \tau}$,

$$\mathbf{z}_t^{(b)} \;=\; \frac{1}{\tau}\,\widetilde{\mathbf{P}}\,\mathbf{s}_t^{(b)} \;\in\; \mathbb{R}^V.$$

Coordinate v of $\mathbf{z}_t^{(b)}$ is the inner product $\langle \widetilde{\mathbf{P}}_v, \mathbf{s}_t^{(b)} \rangle / \tau$ — the alignment of the current state with prototype v.

Step 5 — Loss (training only).

$$\mathcal{L} \;=\; \frac{1}{B\,T} \sum_{b=0}^{B-1} \sum_{t=0}^{T-1} \mathrm{CE}\big(\mathbf{z}_t^{(b)},\, y_t^{(b)}\big).$$

5 Worked training step on a toy instance

Toy configuration. V=4, d=4, T=3, L=1, B=1. Vocabulary: $\{\,0{:}\,\texttt{the},\ 1{:}\,\texttt{cat},\ 2{:}\,\texttt{sat},\ 3{:}\,\texttt{mat}\,\}$. Sequence: $\mathbf{x} = (\texttt{the}, \texttt{cat}, \texttt{sat}) = (0, 1, 2)$, targets $\mathbf{y} = (\texttt{cat}, \texttt{sat}, \texttt{mat}) = (1, 2, 3)$ (next-token).

Initial parameter values

We hand-pick small integers so every step is checkable by mental arithmetic.

$$\mathbf{V} = \begin{pmatrix} +0.4 & -0.2 & +0.7 & -0.1 \\ -0.3 & +0.6 & -0.5 & +0.2 \\ +0.1 & +0.8 & -0.4 & -0.6 \\ +0.5 & -0.7 & +0.3 & +0.9 \end{pmatrix} \;\Rightarrow\; \widetilde{\mathbf{V}} = \widetilde{\mathrm{sign}}\,(\mathbf{V}) = \begin{pmatrix} +1 & -1 & +1 & -1 \\ -1 & +1 & -1 & +1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & +1 & +1 \end{pmatrix}.$$

$$\mathbf{P} \;\Rightarrow\; \widetilde{\mathbf{P}} = \begin{pmatrix} +1 & +1 & -1 & -1 \\ -1 & +1 & +1 & -1 \\ +1 & -1 & +1 & -1 \\ -1 & -1 & -1 & +1 \end{pmatrix}, \qquad \mathbf{m} = \big(+0.5,\ +0.5,\ +0.5,\ +0.5\big) \;\Rightarrow\; \widetilde{\mathbf{m}} = (+1, +1, +1, +1).$$

$$\log \tau = \tfrac{1}{2} \log d = \tfrac{1}{2} \log 4 \approx 0.693, \qquad \tau = \sqrt{d} = 2.$$

5.1 Step 1 — Token lookup and rotation

Position binding by ρ^t

$t = 0$, **token `the` (id 0).** $\widetilde{\mathbf{V}}_0 = (+1, -1, +1, -1)$. ρ^0 is the identity: $\mathbf{h}_0 = (+1, -1, +1, -1)$.

$t = 1$, **token `cat` (id 1).** $\widetilde{\mathbf{V}}_1 = (-1, +1, -1, +1)$. Right-rotate by 1: $(-1, +1, -1, +1) \xrightarrow{\rho^1} (+1, -1, +1, -1)$. So $\mathbf{h}_1 = (+1, -1, +1, -1)$.[a]

$t = 2$, **token `sat` (id 2).** $\widetilde{\mathbf{V}}_2 = (+1, +1, -1, -1)$. Right-rotate by 2: $(+1, +1, -1, -1) \xrightarrow{\rho^2} (-1, -1, +1, +1)$. So $\mathbf{h}_2 = (-1, -1, +1, +1)$.

[a] For d=4, rotation by 1 takes $(a_0, a_1, a_2, a_3) \mapsto (a_3, a_0, a_1, a_2)$.

5.2 Step 2 — Recurrence

With $\widetilde{\mathbf{m}} = (+1, +1, +1, +1)$ the recurrence simplifies to $\mathbf{s}_t = \tanh(\mathbf{s}_{t-1} + \mathbf{h}_t)$.

State evolution

$\mathbf{s}_{-1} = (0, 0, 0, 0)$.

$t = 0$**:** update $= \mathbf{h}_0 = (+1, -1, +1, -1)$.
$\mathbf{s}_0 = \tanh(+1, -1, +1, -1) = (+0.7616,\ -0.7616,\ +0.7616,\ -0.7616)$.

$t = 1$**:** $\mathbf{h}_1 = (+1, -1, +1, -1)$, so $\mathbf{s}_0 + \mathbf{h}_1 = (0.7616{+}1,\ -0.7616{-}1,\ 0.7616{+}1,\ -0.7616{-}1) = (+1.7616,\ -1.7616,\ +1.7616,\ -1.7616)$.
$\mathbf{s}_1 = \tanh(\cdot) = (+0.9434,\ -0.9434,\ +0.9434,\ -0.9434)$.

$t = 2$**:** update $= \mathbf{s}_1 + \mathbf{h}_2 = (0.9434{-}1,\ -0.9434{-}1,\ 0.9434{+}1,\ -0.9434{+}1) = (-0.0566,\ -1.9434,\ +1.9434,\ +0.0566)$.
$\mathbf{s}_2 = \tanh(\cdot) = (-0.0566,\ -0.9601,\ +0.9601,\ +0.0566)$.[a]

[a] $\tanh(0.0566) \approx 0.0565$; $\tanh(1.9434) \approx 0.9601$.

Already we can see what the recurrence does: channels for which $\mathbf{h}_t$ has the same sign across positions accumulate strongly; channels where $\mathbf{h}_t$ flips sign cancel. With $\widetilde{\mathbf{m}} = +1$ everywhere this is "always remember"; in a trained model some m_i are -1, giving "flip every step".

5.3 Step 3 — Logits and loss at the final position

At $t = 2$ the target token is $y_2 = 3$ (`mat`). We compute $\mathbf{z}_2 = \widetilde{\mathbf{P}}\,\mathbf{s}_2/\tau$ with $\tau = 2$.

Per-token logits at t

$\mathbf{s}_2 = (-0.0566,\ -0.9601,\ +0.9601,\ +0.0566)$.

$$\begin{aligned}
\langle \widetilde{\mathbf{P}}_0, \mathbf{s}_2 \rangle &= (+1)(-0.0566) + (+1)(-0.9601) + (-1)(+0.9601) + (-1)(+0.0566) \\
&= -0.0566 - 0.9601 - 0.9601 - 0.0566 \;=\; -2.0334, \\
\langle \widetilde{\mathbf{P}}_1, \mathbf{s}_2 \rangle &= -(-0.0566) + (-0.9601) + (+0.9601) - (+0.0566) \;=\; 0.0000, \\
\langle \widetilde{\mathbf{P}}_2, \mathbf{s}_2 \rangle &= (-0.0566) - (-0.9601) + (+0.9601) - (+0.0566) \;=\; +1.8070, \\
\langle \widetilde{\mathbf{P}}_3, \mathbf{s}_2 \rangle &= -(-0.0566) - (-0.9601) - (+0.9601) + (+0.0566) \;=\; +0.1132.
\end{aligned}$$

Dividing by $\tau = 2$:

$$\mathbf{z}_2 \;=\; (-1.0167,\ 0.0000,\ +0.9035,\ +0.0566).$$

Cross-entropy loss at t

Target is $y_2 = 3$. Apply softmax to $\mathbf{z}_2$:

$$e^{\mathbf{z}_2} = (0.3617,\ 1.0000,\ 2.4683,\ 1.0583), \qquad \sum = 4.8883.$$

$$p_3 = \frac{e^{z_{2,3}}}{\sum} = \frac{1.0583}{4.8883} \approx 0.2165, \qquad \text{CE} = -\log(0.2165) \approx 1.530.$$

A correctly trained model would have made p_3 much larger than the other three; here all four logits are within ≈ 2 of one another, so the loss is large.

5.4 Step 4 — Backward pass through STE

The graph of dependencies for $\mathcal{L}$ on $\mathbf{V}_0$ (the row of `vocab_hv_c` corresponding to token 0, `the`) is:

$$\mathbf{V}_0 \xrightarrow{\widetilde{\text{sign}}} \widetilde{\mathbf{V}}_0 \xrightarrow{\rho^0} \mathbf{h}_0 \xrightarrow{\text{recurrence}} \mathbf{s}_0 \to \mathbf{s}_1 \to \mathbf{s}_2 \xrightarrow{\widetilde{\mathbf{P}}\cdot/\tau} \mathbf{z}_2 \xrightarrow{\text{CE}} \mathcal{L}.$$

The only step that is not analytically differentiable is $\widetilde{\text{sign}}$. By definition (clipped STE),

$$\Big[\nabla_{\mathbf{V}_0}\mathcal{L}\Big]_i = \Big[\nabla_{\widetilde{\mathbf{V}}_0}\mathcal{L}\Big]_i \cdot \mathbf{1}\big[|\mathbf{V}_{0,i}| \le 1\big].$$

Concretely, $\mathbf{V}_0 = (+0.4, -0.2, +0.7, -0.1)$ all have $|\cdot| \le 1$, so the indicator is 1 for every channel and the gradient is passed through unchanged. If at a later training step some $\mathbf{V}_{0,i}$ drifts past ± 1, the gradient for that channel is gated to 0 — preventing runaway away from the bipolar manifold.

What the optimiser actually updates

Adam with $\eta = 3{\cdot}10^{-3}$ takes

$$\mathbf{V} \leftarrow \mathbf{V} - \eta\,\widehat{\nabla_{\mathbf{V}}\mathcal{L}},$$

where the hat is Adam's bias-corrected step. The continuous storage $\mathbf{V}$ drifts slowly; its *sign*, $\widetilde{\mathbf{V}}$, is what the recurrence sees on the next forward and is what gets exported as the 1-bit-per-cell deployment artifact. Sign flips of $\mathbf{V}_{v,i}$ are the only events that change behaviour.

5.5 Recap of the full training step

1. Forward materialises $\widetilde{\mathbf{V}}, \widetilde{\mathbf{P}}, \widetilde{\mathbf{m}}$ via $\widetilde{\text{sign}}$.
2. Tokens are looked up and rotated, producing $\mathbf{h}_0, \mathbf{h}_1, \mathbf{h}_2$.
3. The recurrence produces continuous states $\mathbf{s}_0, \mathbf{s}_1, \mathbf{s}_2$.
4. Each $\mathbf{s}_t$ is dotted with $\widetilde{\mathbf{P}}$ and divided by τ to get logits.
5. Cross-entropy against $\mathbf{y}$ is the loss.
6. Backward: gradients flow through tanh, the inner product, and the clipped STE; only continuous shadows $(\mathbf{V}, \mathbf{P}, \mathbf{m}, \log\tau)$ get gradient updates.

6 Worked inference (autoregressive sampling)

Inference uses the same forward pass, with three differences:

1. No targets, so no loss; we stop at logits.
2. The bipolar tensors come from `.sign()`, not `ste_sign`; the gradient path does not exist. The values are identical.
3. Each new token is sampled and fed back as x_{t+1}; the loop continues for `max_new_tokens` steps.

Toy setting. Same parameters as §5. Prompt: a single token, `the` (id 0). Generate one new token using *argmax* (deterministic, equivalent to top-1).

Forward on the single-token prompt

$\mathbf{s}_{-1} = (0, 0, 0, 0)$, $\mathbf{h}_0 = (+1, -1, +1, -1)$.
$\mathbf{s}_0 = \tanh(\mathbf{h}_0) = (+0.7616,\ -0.7616,\ +0.7616,\ -0.7616)$.

Logits at the first generation step

$$\langle \widetilde{\mathbf{P}}_0, \mathbf{s}_0 \rangle = (+1)(+0.7616) + (+1)(-0.7616) + (-1)(+0.7616) + (-1)(-0.7616) = 0.0000,$$
$$\langle \widetilde{\mathbf{P}}_1, \mathbf{s}_0 \rangle = -0.7616 - 0.7616 + 0.7616 + 0.7616 = 0.0000,$$
$$\langle \widetilde{\mathbf{P}}_2, \mathbf{s}_0 \rangle = +0.7616 + 0.7616 + 0.7616 + 0.7616 = +3.0464,$$
$$\langle \widetilde{\mathbf{P}}_3, \mathbf{s}_0 \rangle = -0.7616 + 0.7616 - 0.7616 - 0.7616 = -1.5232.$$

$\mathbf{z}_0 = (0,\ 0,\ +1.5232,\ -0.7616)$ after $\tau = 2$.

Sampling decision

Argmax over $\mathbf{z}_0$ picks index 2 $\Rightarrow$ token `sat`.

Top-k with k=2, temperature T_s=0.7 would instead form $p \propto \exp(\mathbf{z}_0/T_s)$ restricted to the top-2 indices $\{0, 2\}$ (the others' probabilities are set to 0 before normalisation):

$$\exp(0/0.7) = 1, \quad \exp(1.5232/0.7) \approx 8.846, \qquad p_0 \approx 0.102, \ \ p_2 \approx 0.898.$$

With a random draw $u \sim \mathcal{U}[0, 1]$ from a seeded RNG, we emit `sat` unless $u < 0.102$, in which case we emit `the`.

Continuing the loop. The sampled token is appended to the input and the forward repeats with $T' = 2$ (positions 0 and 1). Because the recurrence is left-to-right and stateful, an efficient implementation caches $\mathbf{s}_{T'-1}$ and runs the recurrence one step. The 6502 firmware does exactly this: it keeps a 256-byte state register and updates it once per emitted token.

7 What the 6502 actually computes

At deployment the math reduces to integer bit operations.

Bipolar dot product as XOR + popcount

For $a, b \in \{-1, +1\}^d$, encode $-1 \mapsto 0$ and $+1 \mapsto 1$, giving $\bar{a}, \bar{b} \in \{0, 1\}^d$. Then

$$\langle a, b \rangle \ = \ d - 2 \cdot \mathrm{popcount}(\bar{a} \oplus \bar{b}),$$

where $\oplus$ is bitwise XOR. The mapping is exact: agreements add +1 each, disagreements add -1 each.

Inference loop in bit-arithmetic terms

At each emitted token:

1. **Token lookup:** fetch the $d/8$ bytes of $\widetilde{\mathbf{V}}_{x_t}$ from EEPROM.

2. **Rotation:** cyclic shift those bytes by t bits.
3. **Recurrence:** for each of d channels, compute $\widetilde{m}_i \cdot s_{i,t-1} + h_{i,t}$ in fixed-point and apply a tanh LUT.
4. **Logits:** for each of V rows of $\widetilde{\mathbf{P}}$, XOR with the thresholded state and popcount; convert to a fixed-point logit.
5. **Sample:** softmax + top-k + seeded PRNG.

The two storage-heavy tables $(\widetilde{\mathbf{V}}, \widetilde{\mathbf{P}})$ account for 16,384 of the 16,434 bytes; everything else is scratch.

8 Cross-reference to the source

Every equation above corresponds to a specific line range:

Concept	File	Lines
$\widetilde{\text{sign}}$ definition	`wozformer/ste.py`	15–24
Parameter declarations	`wozformer/models/hdc_rwkv.py`	30–36
Forward (training, STE)	`wozformer/models/hdc_rwkv.py`	39–74
Forward (deployment, `.sign()`)	`wozformer/models/hdc_rwkv.py`	78–111
Deployment byte counter	`wozformer/models/hdc_rwkv.py`	114–117

The block diagram in `architecture_hdc_rwkv.tex` visualises the same flow at the shipping dimensions V=256, d=256. Only the magnitudes change; the math is identical.